

SIMATS ENGINEERING

ASSESSMENT TOOL III – VISUALIZATION CRITIQUE & REDESIGN

Course: Data Handling and Visualization	Code: DSA0601	CO: CO3
Duration: 120 min	Total Marks: 100	Reg No: 192324285
Name: Surya JK		

COURSE OUTCOME COVERED

CO3: Analyze time-series data, trends, associations, and uncertainty through appropriate visualization methods. (BL4)

Activity Tool: Visualization Critique & Redesign

Weightage: 20%

Code of Conduct:

I Surya JK / 192324285 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Criteria	Evaluation of Existing Visualization (25%)	Redesign Strategy (25%)	Application of Vis. Principles (20%)	Organization (15%)	Accuracy (10%)	Clarity (5%)	Total
Q1							
Q2							
Q3							
Q4							
Q5							

Question 1 – Critique and Redesign an ECDF and Q-Q Plot

Dataset: *Delivery_Time_Distribution.csv* | Warehouses: Chennai, Bengaluru, Hyderabad, Pune

1. R Code – Reproducing the Original (Problematic) Visualization

```
df <- read.csv("Delivery_Time_Distribution.csv")
df$Warehouse <- factor(df$Warehouse,
  levels = c("Chennai", "Bengaluru", "Hyderabad", "Pune"))

par(mfrow = c(1,2))

# Bad ECDF: no shared x-axis calibration, cluttered grid, no reference
plot(NULL, xlim = c(0,25), ylim = c(0,1),
  xlab = "Delivery Time", ylab = "Fn(x)",
  main = "ECDF of Delivery Time (by Warehouse)")
grid(nx = 20, ny = 20, col = "gray70", lty = 1)
cols <- c("red", "blue", "green3", "orange")
for (i in seq_along(levels(df$Warehouse))) {
  w <- levels(df$Warehouse)[i]
  x <- sort(df$Delivery_Time_Hrs[df$Warehouse == w])
  lines(ecdf(x), col = cols[i], lwd = 1, verticals = TRUE, do.points = TRUE)
}
legend("bottomright", legend = levels(df$Warehouse), col = cols, lty = 1)

# Bad Q-Q plot: pooled data, no reference line, cluttered grid
qqnorm(df$Delivery_Time_Hrs, main = "Q-Q Plot (Pooled)")
grid(nx = 20, ny = 20, col = "gray70", lty = 1)
points(qqnorm(df$Delivery_Time_Hrs, plot.it = FALSE), col = "purple", pch = 16)
```



Figure 1.1 – Original ECDF and Q-Q plot: cluttered gridlines, no shared axis calibration, pooled data, no reference line.

2. Identified Visualization Problems (at least 3)

- Inconsistent / uncalibrated axis scaling – the ECDF panel is not scaled to a single, deliberately chosen range for all four warehouses, so visual comparison of curve steepness across warehouses is unreliable.
- Missing reference line – neither plot has a reference (a theoretical normal ECDF overlay, or a qqline in the Q-Q plot), so the viewer cannot judge deviation from an expected distribution at a glance.
- Cluttered gridlines – dense default 20x20 gridlines visually compete with the data, making it hard to trace individual ECDF steps or isolate Q-Q points.
- Pooled Q-Q plot – the Q-Q plot ignores the Warehouse grouping entirely, which can mask warehouse-specific skewness and outliers (e.g. Pune's outlier is diluted into the pooled sample).
- Overplotted legend/colors – four overlapping step lines on a single small panel make individual warehouse curves difficult to distinguish.

3. R Code – Redesigned Visualization

```
library(ggplot2)

# Redesigned ECDF: common x-axis, minimal theme, per-warehouse color
```

```
p1 <- ggplot(df, aes(Delivery_Time_Hrs, color = Warehouse)) +
  stat_ecdf(geom = "step", linewidth = 0.9) +
  scale_x_continuous(limits = c(0,24), breaks = seq(0,24,4)) +
  scale_color_brewer(palette = "Set2") +
  labs(title = "ECDF of Delivery Time by Warehouse",
       x = "Delivery Time (hrs)", y = "Cumulative Proportion F(x)") +
  theme_minimal(base_size = 13) +
  theme(panel.grid.minor = element_blank())

# Redesigned Q-Q plot: faceted per warehouse with reference line
p2 <- ggplot(df, aes(sample = Delivery_Time_Hrs)) +
  stat_qq(color = "#2c7fb8", size = 2) +
  stat_qq_line(color = "firebrick", linewidth = 0.8) +
  facet_wrap(~Warehouse, nrow = 1) +
  labs(title = "Q-Q Plot of Delivery Time by Warehouse",
       x = "Theoretical Quantiles", y = "Sample Quantiles") +
  theme_minimal(base_size = 12) +
  theme(panel.grid.minor = element_blank())
```

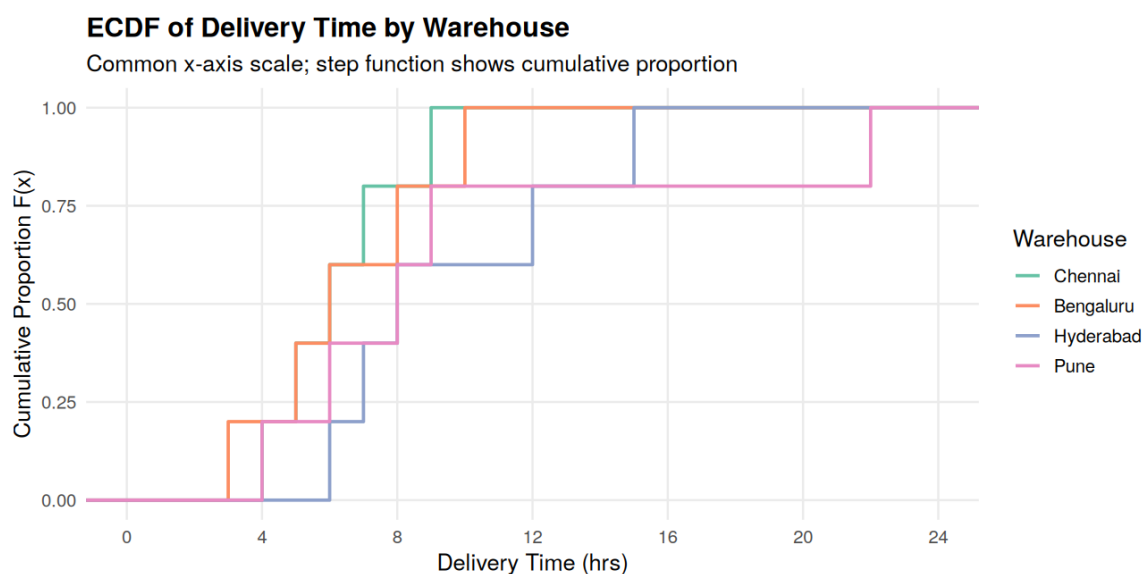


Figure 1.2 – Redesigned ECDF: shared x-axis scale, minimal gridlines, clean legend.

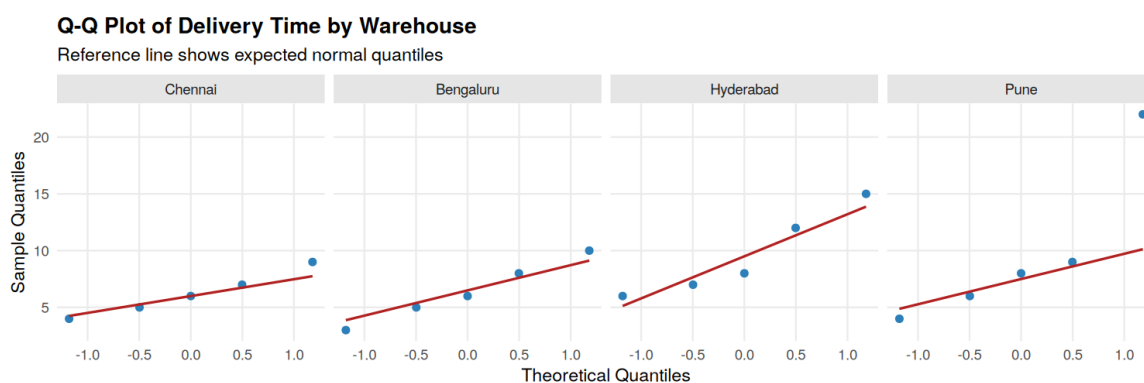


Figure 1.3 – Redesigned Q-Q plot faceted by warehouse with a normal reference line.

4. How the Redesign Improves Interpretation

Skewness: In the redesigned ECDF, the steepness and position of each warehouse's step curve on a common 0–24 hr axis makes right-skew immediately visible — Hyderabad and Pune both show curves that rise quickly at low values and then stretch out with a long flat-to-vertical tail, indicating a right-skewed distribution with a few very slow deliveries.

Outliers: In the faceted Q-Q plots, the added reference (qqline) makes deviation obvious — Pune's 22-hour delivery sits far above the line at the top-right, and Hyderabad's 15-hour delivery similarly departs from the line, flagging both as clear outliers. This was impossible to see reliably in the pooled, unreferenced original.

Normality: Points that hug the red reference line (Chennai, Bengaluru) indicate delivery times closer to a normal distribution, while points that curve away from the line (Hyderabad, Pune) indicate departure from normality caused by the long right tail.

5. Is an ECDF Plot Alone Sufficient to Detect Outliers?

No. An ECDF can hint at an outlier through a large, isolated vertical jump near the top of the curve, but this signal is easy to miss, especially when several groups are overlaid or gridlines are cluttered. An ECDF should be paired with a complementary plot — a boxplot (which explicitly flags points beyond $1.5 \times \text{IQR}$) or a Q-Q plot (which shows deviation from an expected distribution) — to reliably identify and confirm outliers such as Pune's 22-hour delivery.

Question 2 – Critique and Redesign Many Distributions Using Bean Plots

Dataset: *Subject_Marks.csv* | Subjects: *Physics, Chemistry, Mathematics, Biology*

1. R Code – Reproducing the Original (Problematic) Visualization

```
df <- read.csv("Subject_Marks.csv")
subs <- c("Physics", "Chemistry", "Mathematics", "Biology")

par(mfrow = c(1,4))
# deliberately different / truncated y-limits per subject
ylims <- list(Physics = c(55,88), Chemistry = c(50,85),
             Mathematics = c(58,92), Biology = c(53,90))
for (s in subs) {
  boxplot(df$Marks[df$Subject == s], main = s, ylab = "Marks",
          ylim = ylims[[s]], col = "lightblue")
}
```

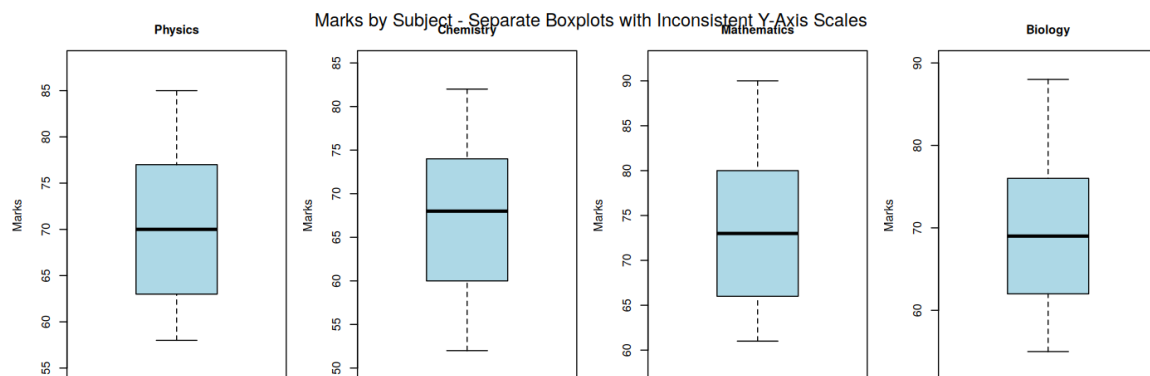


Figure 2.1 – Four separate boxplots, each on its own truncated y-axis range.

2. Critique

- Inconsistent y-axis scaling: each panel uses a different range (e.g. Physics 55–88 vs Mathematics 58–92), so box heights are not visually comparable across subjects even though the underlying spreads differ by only a few marks.
- Truncated axes: none of the axes start from a common baseline, which exaggerates apparent differences in the central tendency between subjects.
- Four disconnected panels force the reader to jump between plots and mentally re-align scales rather than compare directly within a single frame.
- Boxplots alone hide the underlying density shape (e.g. multimodality or skew within a subject), since only five summary numbers are shown per subject.

3. R Code – Redesign Visualization (Violin / Bean-style Plot, Common Scale)

```
library(ggplot2)
library(vioplot)

# Redesign 1: ggplot violin + boxplot overlay, single common y-axis
p <- ggplot(df, aes(Subject, Marks, fill = Subject)) +
  geom_violin(trim = FALSE, alpha = 0.6) +
  geom_boxplot(width = 0.12, fill = "white", alpha = 0.9) +
  geom_jitter(width = 0.05, size = 1.6, alpha = 0.6) +
  scale_y_continuous(limits = c(45,95), breaks = seq(45,95,10)) +
  labs(title = "Marks Distribution by Subject",
       x = "Subject", y = "Marks") +
  theme_minimal(base_size = 13) + theme(legend.position = "none")

# Redesign 2: base-R bean/violin-style plot, common axis
vioplot(Marks ~ Subject, data = df,
        ylim = c(45,95),
        main = "Bean/Violin-style Plot of Marks by Subject",
        xlab = "Subject", ylab = "Marks")
```

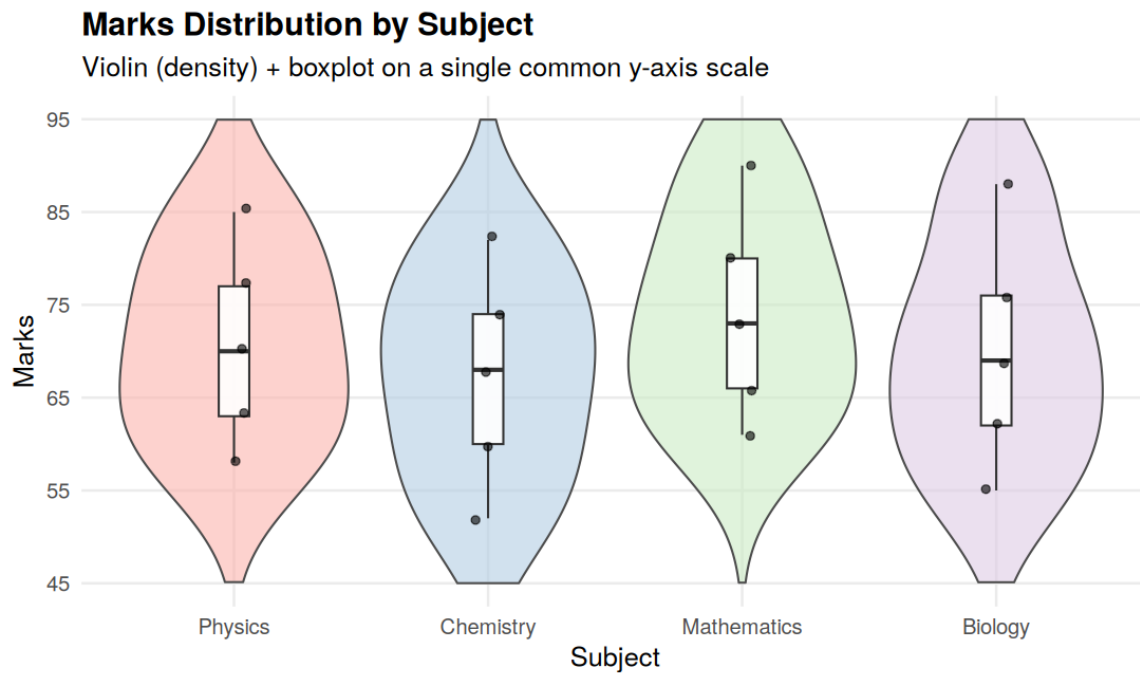


Figure 2.2 – Redesigned violin + boxplot + jittered points on one common axis (45–95).

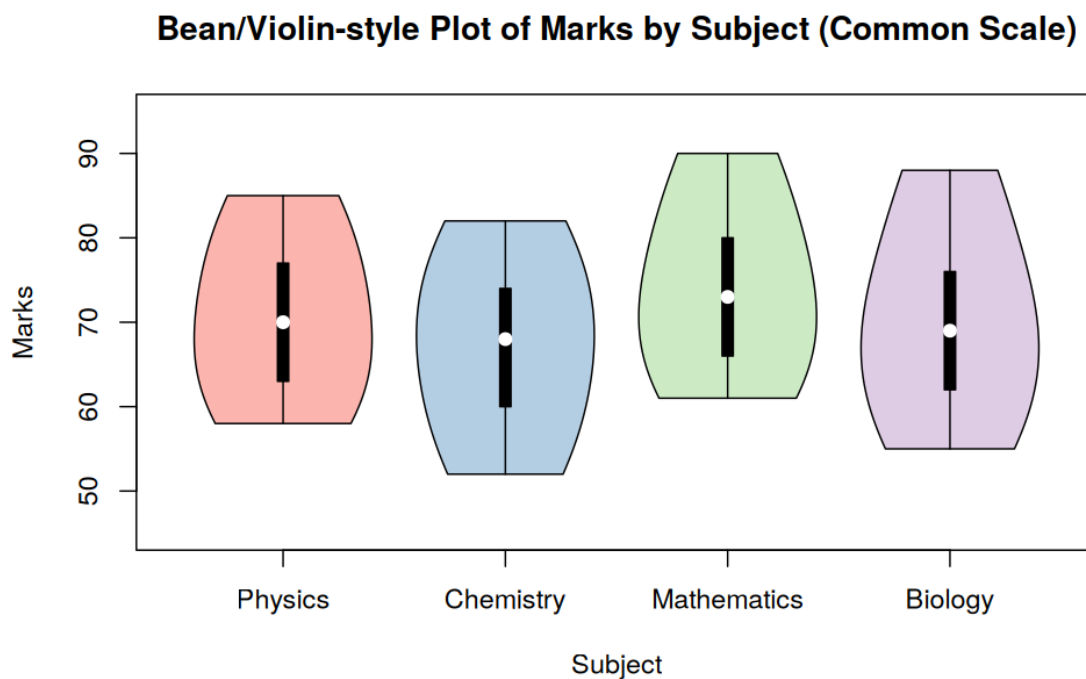


Figure 2.3 – Bean/violin-style plot (vioplot) as a base-R alternative, also on a common axis.

4. Common Scale and Labels

Both redesigns place all four subjects on the same y-axis (45–95, breaks of 10) within a single panel, with clear subject labels on the x-axis. This lets the eye compare medians, spreads, and shapes directly without re-reading axis numbers for each subject.

5. Concentration and Spread — Answer

Subject	Mean	Std. Dev.	Range
Physics	70.6	10.8	27
Chemistry	67.2	11.7	30
Mathematics	74.0	11.5	29
Biology	70.0	12.7	33

Physics shows the highest concentration of marks — its distribution has the smallest standard deviation (10.8) and range (27), meaning student scores cluster most tightly around the mean (70.6), visible in Figure 2.2/2.3 as the narrowest violin shape.

Biology shows the greatest spread of marks — it has the largest standard deviation (12.7) and the widest range (33, from 55 to 88), visible as the widest/tallest violin shape stretching furthest from its median.

Question 3 – Critique and Redesign Proportion Visualizations

Dataset: App_Downloads.csv | Cities: Chennai, Mumbai, Delhi, Kolkata

1. R Code – Reproducing the Original (Problematic) Visualization

```
df <- read.csv("App_Downloads.csv")
cities <- c("Chennai", "Mumbai", "Delhi", "Kolkata")

# deliberately mismatched color palettes per city, no shared legend
palettes <- list(Chennai = c("red", "gold", "cyan"),
                  Mumbai = c("purple", "green", "pink"),
                  Delhi = c("orange", "blue", "brown"),
                  Kolkata = c("darkred", "yellow", "grey"))

par(mfrow = c(2,2))
for (city in cities) {
  sub <- df[df$City == city, ]
  pie(sub$Downloads, labels = paste0(sub$App_Category, "\n", sub$Downloads),
      col = palettes[[city]], main = city, init.angle = 90)
}
```

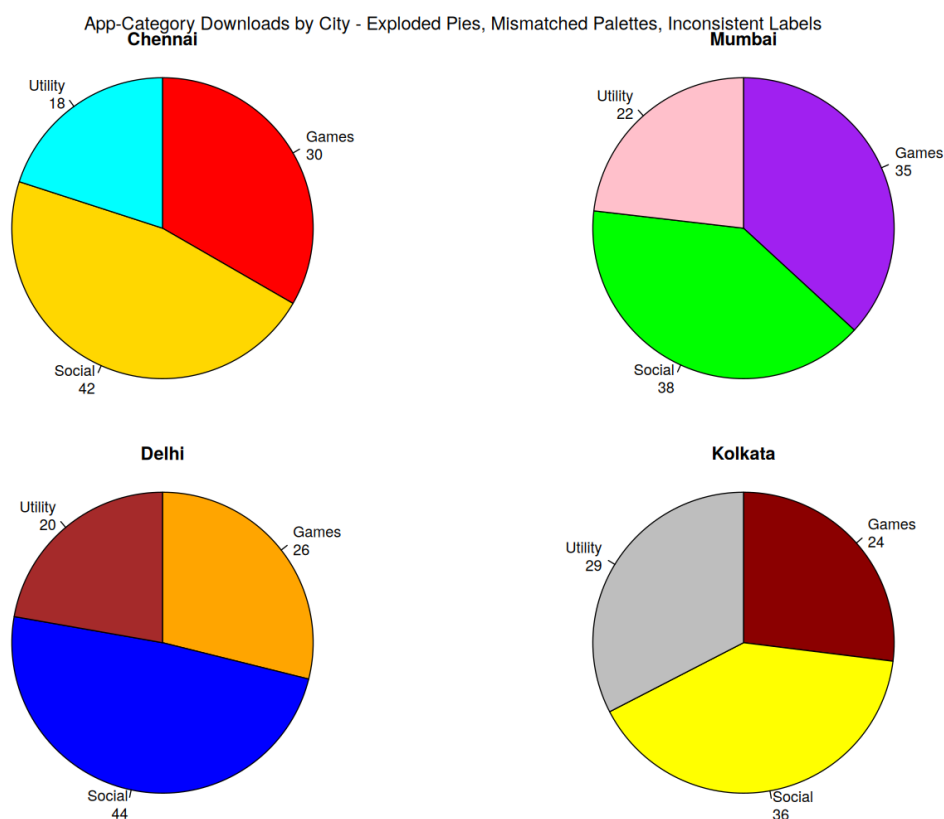


Figure 3.1 – Four pie charts, one per city, with mismatched color palettes and inconsistent labeling.

2. Design Problems (at least 4)

- Mismatched color palettes across the four city pies — the same category (e.g. 'Games') is drawn in a different color in every city, breaking the mental color-to-category mapping and making cross-city comparison error-prone.
- Pie charts require angle/area judgments, which the human eye estimates far less accurately than the length judgments used in bar charts — small differences (e.g. Games 30 vs Utility 18 in Chennai) are harder to compare precisely.
- Inconsistent labeling — raw counts are shown without any indication of each city's total, so a slice's absolute size cannot be related to relative share without extra mental math.
- Four separate, disconnected charts force the reader to read each pie individually and hold four sets of numbers in memory to compare cities, rather than comparing directly within one frame.
- No shared legend or consistent slice ordering, so the starting angle and arrangement of categories differs from city to city, further slowing comparison.

3. R Code – Redesign 1: Side-by-Side (Grouped) Bar Chart

```
library(ggplot2)
ggplot(df, aes(City, Downloads, fill = App_Category)) +
  geom_col(position = position_dodge(width = 0.75), width = 0.65) +
  scale_fill_brewer(palette = "Set2") +
  labs(title = "App Downloads by City and Category",
       x = "City", y = "Downloads", fill = "App Category") +
  theme_minimal(base_size = 13)
```

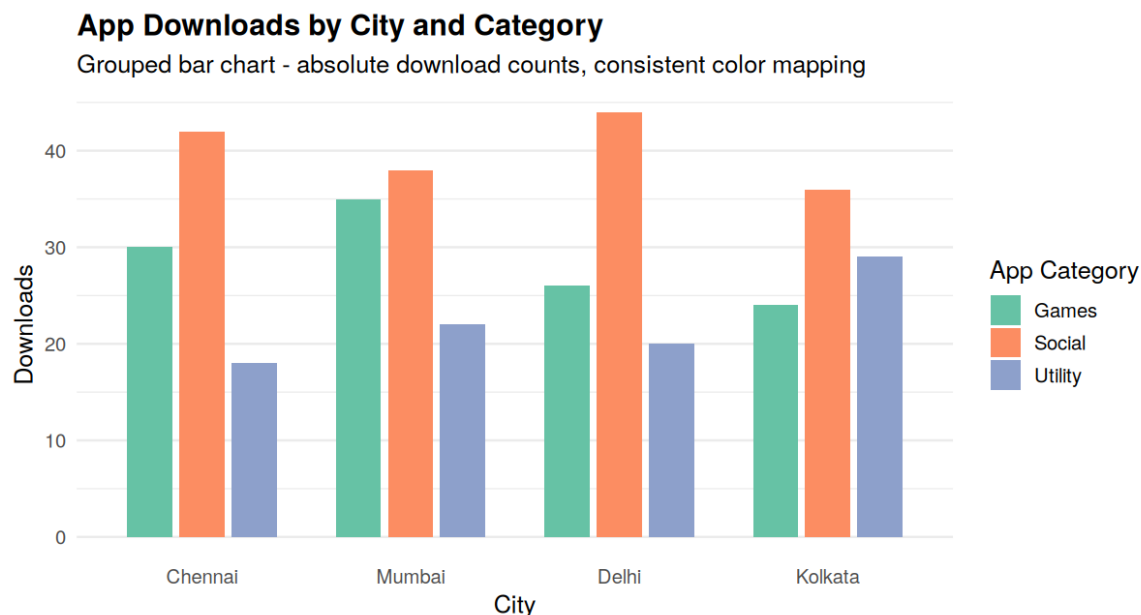


Figure 3.2 – Redesign 1: grouped bar chart with one consistent color per category.

4. R Code – Redesign 2: 100% Stacked Bar Chart

```
library(dplyr)
df2 <- df %>% group_by(City) %>% mutate(pct = Downloads/sum(Downloads)*100)

ggplot(df2, aes(City, pct, fill = App_Category)) +
  geom_col(width = 0.6, position = "fill") +
  scale_y_continuous(labels = scales::percent_format(scale = 1)) +
  scale_fill_brewer(palette = "Set2") +
  labs(title = "Proportion of App-Category Downloads by City",
       x = "City", y = "Share of Downloads", fill = "App Category") +
  theme_minimal(base_size = 13)
```

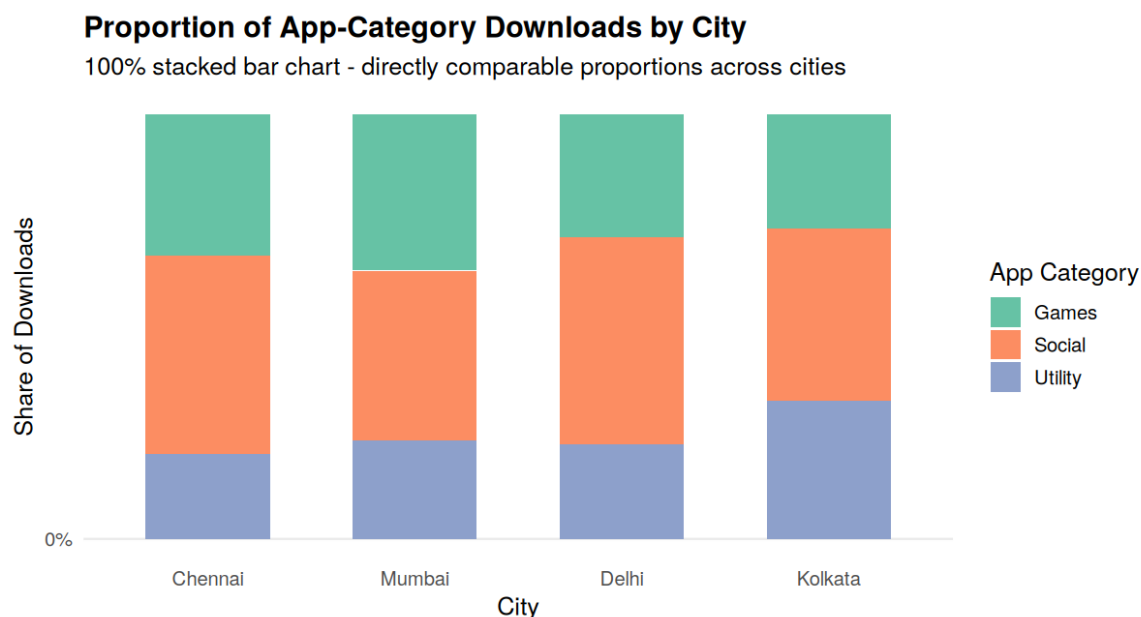


Figure 3.3 – Redesign 2: 100% stacked bar chart showing directly comparable proportions.

5. Comparing the Two Redesigns

For comparing absolute download volumes, the grouped bar chart (Figure 3.2) is preferable since bar length directly encodes the raw count for each category in each city. For comparing city-wise proportions specifically — which is the marketing team's stated goal — the 100% stacked bar chart (Figure 3.3) is the better choice: every bar is normalized to the same 100% baseline, so each segment's height reads directly as a percentage share, letting the viewer instantly compare, for example, Social's share in Delhi (~49%) versus Chennai (~47%) without any mental division.

6. Why Exploded Pies and Mismatched Palettes Mislead

Exploding a pie slice pulls it visually away from the center, which increases its perceived area and prominence relative to its true numeric share — since pie interpretation already relies on the less-accurate skill of angle/area judgment, this exaggeration compounds the distortion. Mismatched color palettes across charts defeat consistent pattern recognition: the viewer must re-learn which color represents which category on every chart, which slows comparison and increases the chance of misreading one category for another when scanning quickly across the four cities.

Question 4 – Critique and Redesign Nested Proportions

Dataset: *Campus_Expenditure.csv* | Hierarchy: *Campus -> Department -> Expense Type*

1. R Code – Reproducing the Original (Problematic) Treemap

```
library(ggplot2)
# rects: pre-computed treemap rectangles (Campus -> Department -> Expense_Type)
# original: continuous grey-scale fill tied only to Amount, tiny labels, no borders
ggplot(rects) +
  geom_rect(aes(xmin = xmin, xmax = xmax, ymin = ymin, ymax = ymax,
               fill = Amount), color = "white", linewidth = 0.3) +
  geom_text(aes(x = cx, y = cy, label = Expense_Type), size = 2.0) +
  scale_fill_gradient(low = "grey90", high = "grey30") +
  labs(title = "Campus Expenditure Treemap (Original)") +
  theme_void()
```

Campus Expenditure Treemap (Original)

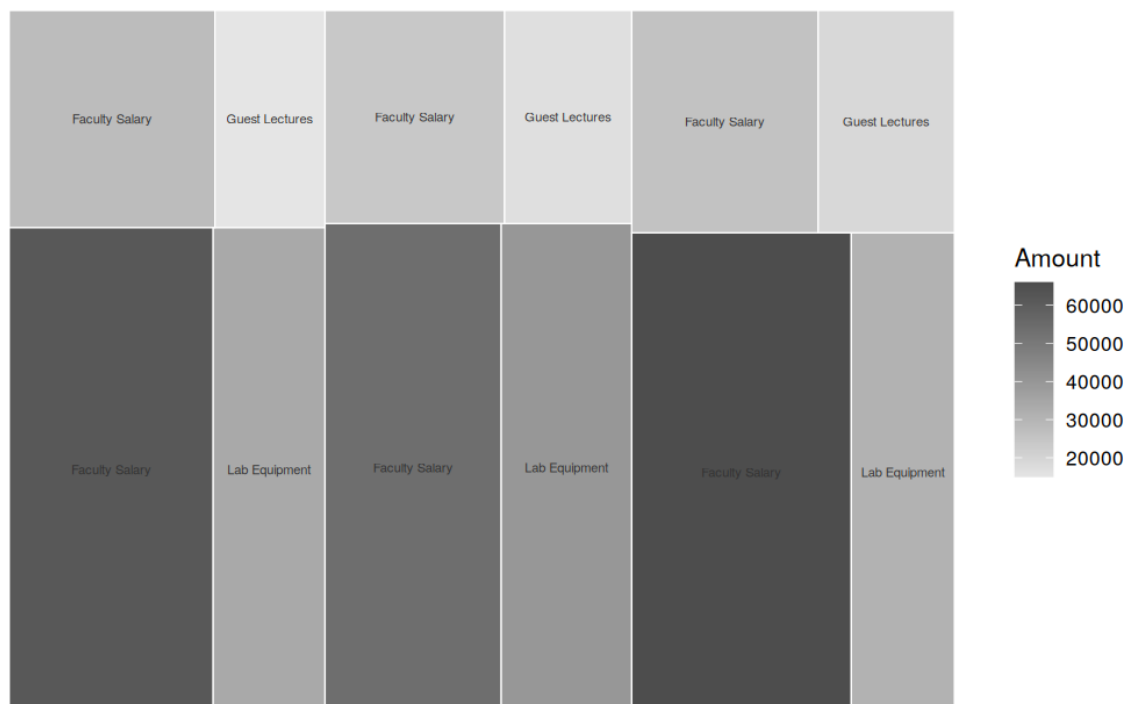


Figure 4.1 – Original treemap: unreadable small labels, no hierarchy borders, weak grey color scale.

2. Critique

- Size: rectangle areas are proportional to Amount, but with no visual grouping the viewer cannot tell which rectangles belong to the same Campus or Department — the hierarchy encoded in the data is lost in the layout.
- Labeling: text size does not adapt sensibly to rectangle size — labels for smaller expense items (e.g. Guest Lectures) are cramped and hard to read, and no amount values are shown.
- Color: a single continuous grey gradient mapped to Amount does not distinguish Campus from Department — two different rectangles from different campuses can appear the same shade purely by coincidence of value, actively working against hierarchy comprehension.
- Hierarchy: there are no borders or grouping cues separating the three campuses, so the chart reads as 12 flat, unrelated rectangles rather than a 3-level nested structure.

3. R Code – Redesigned Treemap

```
# redesign: categorical fill by Department, thick black border per Campus,
# readable bold labels with Expense Type + Amount, Campus header above each group
ggplot() +
  geom_rect(data = rects, aes(xmin, xmax, ymin, ymax, fill = Department),
           color = "white", linewidth = 1.1) +
  geom_rect(data = camp_bounds, aes(xmin, xmax, ymin, ymax),
           fill = NA, color = "black", linewidth = 1.3) +
```

```
geom_text(data = rects, aes(cx, cy, label = paste0(Expense_Type, "\nRs. ", Amount)),
  size = 3.1, fontface = "bold") +
geom_text(data = camp_bounds, aes((xmin+xmax)/2, ymax + 3, label = Campus),
  size = 4.5, fontface = "bold") +
scale_fill_brewer(palette = "Set2") +
labs(title = "Campus Expenditure Treemap (Redesigned)") +
theme_void()
```

Campus Expenditure Treemap (Redesigned)

Outer border = Campus, color = Department, label = Expense Type & Amount

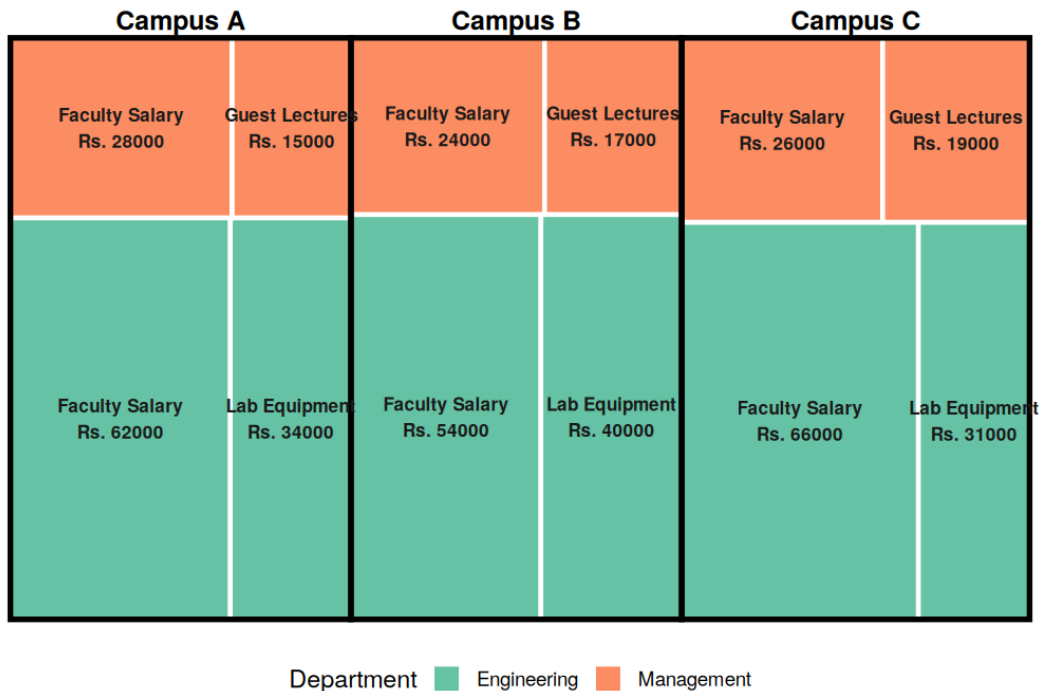


Figure 4.2 – Redesigned treemap: black borders group each Campus, color encodes Department, labels show Expense Type and Amount.

4. R Code – Sunburst Alternative

```
# sunburst: three concentric rings (Campus, Department, Expense Type)
# built as a stacked polar bar chart
ggplot() +
  geom_col(data = ring1, aes(x = 1, y = val, fill = Campus), color = "white") +
  geom_col(data = ring2, aes(x = 2, y = val, fill = Campus), color = "white", alpha = 0.75) +
  geom_col(data = ring3, aes(x = 3, y = val, fill = Campus), color = "white", alpha = 0.5) +
  coord_polar(theta = "y") +
  labs(title = "Campus Expenditure Sunburst Alternative") +
  theme_void()
```

Campus Expenditure Sunburst Alternative

Inner ring = Campus, middle = Department, outer = Expense Type

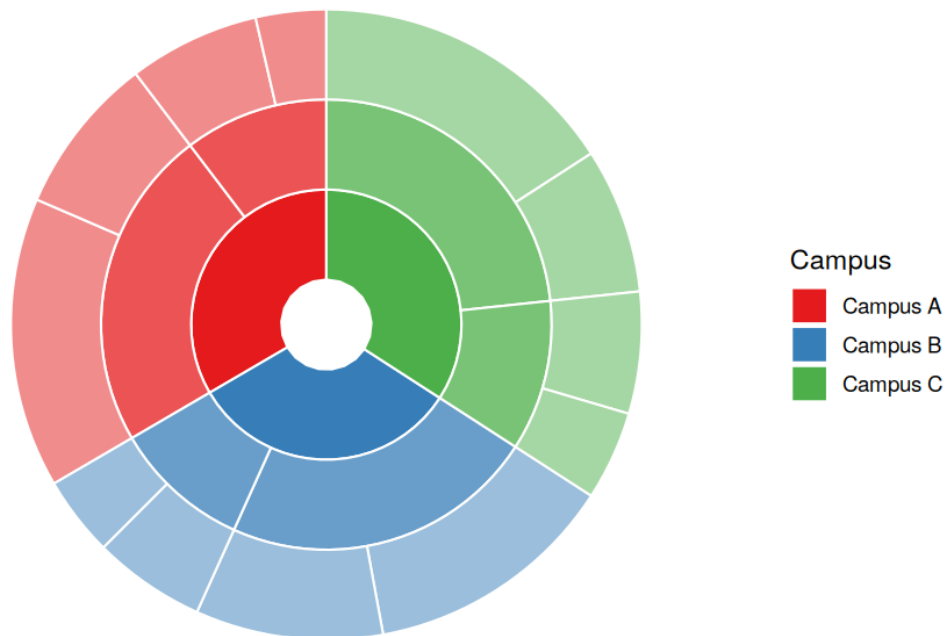


Figure 4.3 – Sunburst alternative: inner ring = Campus, middle = Department, outer = Expense Type.

5. Comparing the Redesigned Treemap and the Sunburst

The treemap encodes magnitude with rectangle area, which research on graphical perception (Cleveland & McGill) shows people judge more accurately than the angular/arc-length encoding used by the sunburst. The sunburst, however, makes the parent-child containment relationship more visually intuitive at a glance (rings nest naturally), and can look more engaging in a presentation, but comparing two arc lengths in the outer ring is harder than comparing two rectangle areas, especially when they are not adjacent.

6. Recommendation for a Board Presentation

The redesigned treemap is recommended for a board presentation. Board members need to quickly and accurately compare expenditure amounts across campuses and departments; the treemap's area-based encoding supports more precise magnitude comparison than the sunburst's angular encoding, and the explicit campus borders make the hierarchy legible without requiring the audience to interpret an unfamiliar circular layout — an important consideration for a non-technical executive audience under time pressure.

Question 5 – Critique and Redesign Flow-Based Proportions

Dataset: *Course_to_Career_Flow.csv* | Flow: *Course -> Certification -> Job Role*

1. R Code – Reproducing the Original (Problematic) Sankey Diagram

```
library(ggalluvial)
df <- read.csv("Course_to_Career_Flow.csv")
df_long <- df %>% to_lodes_form(axes = 1:3, id = "Cohort")

# original: near-identical blue shades for every course, no node/flow labels
ggplot(df_long, aes(x = x, stratum = stratum, alluvium = Cohort, y = Students)) +
  geom_alluvium(aes(fill = Course), color = NA) +
  geom_stratum(fill = "grey80", color = "grey60", width = 1/4) +
  scale_fill_manual(values = c("#5B8DB8", "#5C90BA", "#5A8FB9", "#5D91BB")) +
  labs(title = "Course -> Certification -> Job Role (Original Sankey)") +
  theme_minimal()
```

Course -> Certification -> Job Role (Original Sankey)

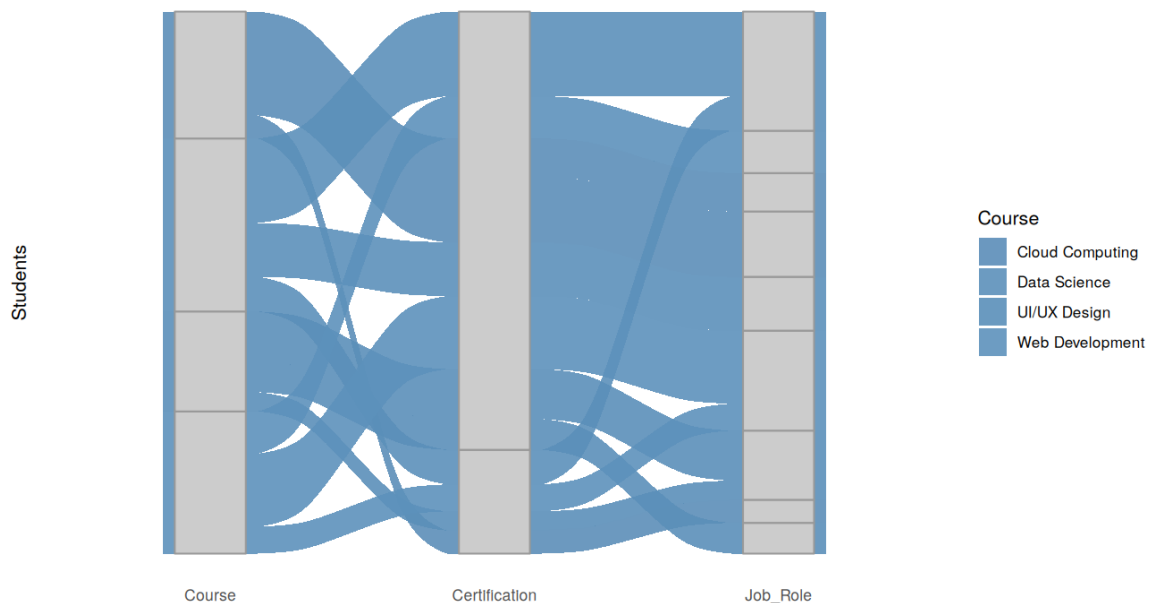


Figure 5.1 – Original Sankey: near-identical color shades and unlabelled nodes/flows.

2. Sources of Confusion / Misleading Interpretation (at least 4)

- Near-identical shades of blue are used for all four courses, making it almost impossible to tell which course a given flow band originates from.
- No labels on the Certification or Job Role nodes, so the viewer cannot tell which stratum block corresponds to which category without guessing from position alone.
- No student-count labels on the flows, so relative flow widths must be judged purely by eye, which is unreliable for close values (e.g. 22 vs 19 students).
- Node bar widths are not annotated, so even though width is proportional to total students, the chart provides no way to read off the actual totals.
- Heavy crossing of flow bands in the middle (Certification) stage, with no ordering or transparency variation to help the eye trace an individual path from Course to Job Role.

3. R Code – Redesigned Sankey Diagram

```
# redesign: distinct qualitative color per Course, labeled strata, subtitle
ggplot(df_long, aes(x = x, stratum = stratum, alluvium = Cohort, y = Students)) +
  geom_alluvium(aes(fill = Course), color = "white", linewidth = 0.15) +
  geom_stratum(fill = "white", color = "grey30", width = 1/4) +
  geom_text(stat = "stratum", aes(label = after_stat(stratum)), fontface = "bold") +
  scale_fill_brewer(palette = "Set1") +
  scale_x_discrete(labels = c("Course", "Certification", "Job Role")) +
  labs(title = "Course -> Certification -> Job Role (Redesigned Sankey)",
       y = "Number of Students", fill = "Course") +
  theme_minimal(base_size = 12)
```

Course -> Certification -> Job Role (Redesigned Sankey)

Distinct color per course, labeled nodes, flow width = number of students

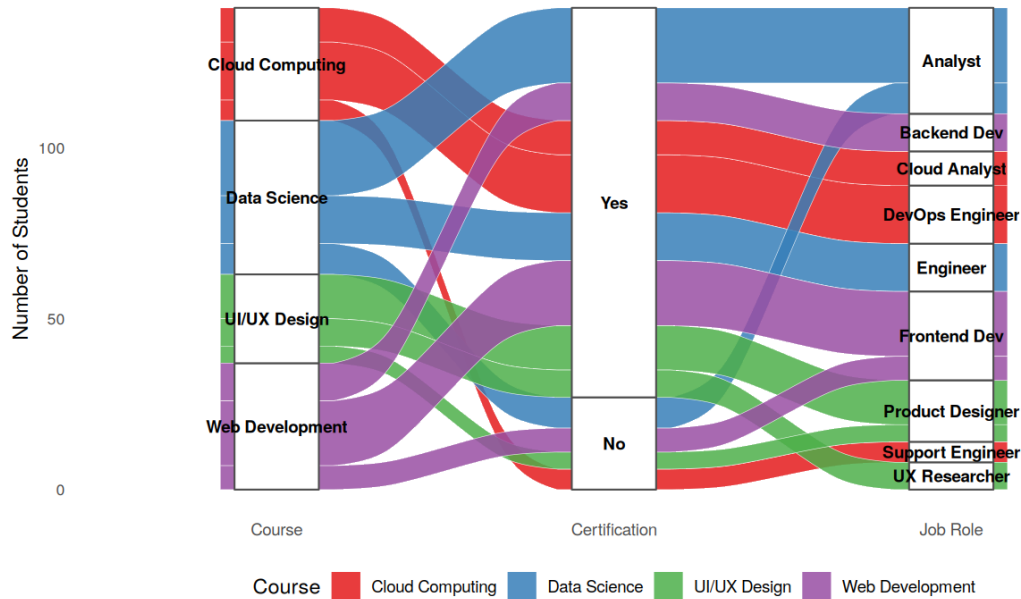


Figure 5.2 – Redesigned Sankey: distinct course colors, labeled nodes, flow width = student count.

4. R Code – Parallel Sets Alternative

```
# parallel sets: same alluvial data, wider strata blocks, in-block labels
ggplot(df_long, aes(x = x, stratum = stratum, alluvium = Cohort,
                    y = Students, fill = Course, label = stratum)) +
  geom_flow(stat = "alluvium", alpha = 0.7, color = "white") +
  geom_stratum(width = 0.28, color = "grey20") +
  geom_text(stat = "stratum", size = 3.2) +
  scale_fill_brewer(palette = "Set1") +
  labs(title = "Parallel Sets: Course -> Certification -> Job Role") +
  theme_minimal(base_size = 12)
```

Parallel Sets: Course -> Certification -> Job Role

Alternative layout emphasising categorical strata over flow curvature

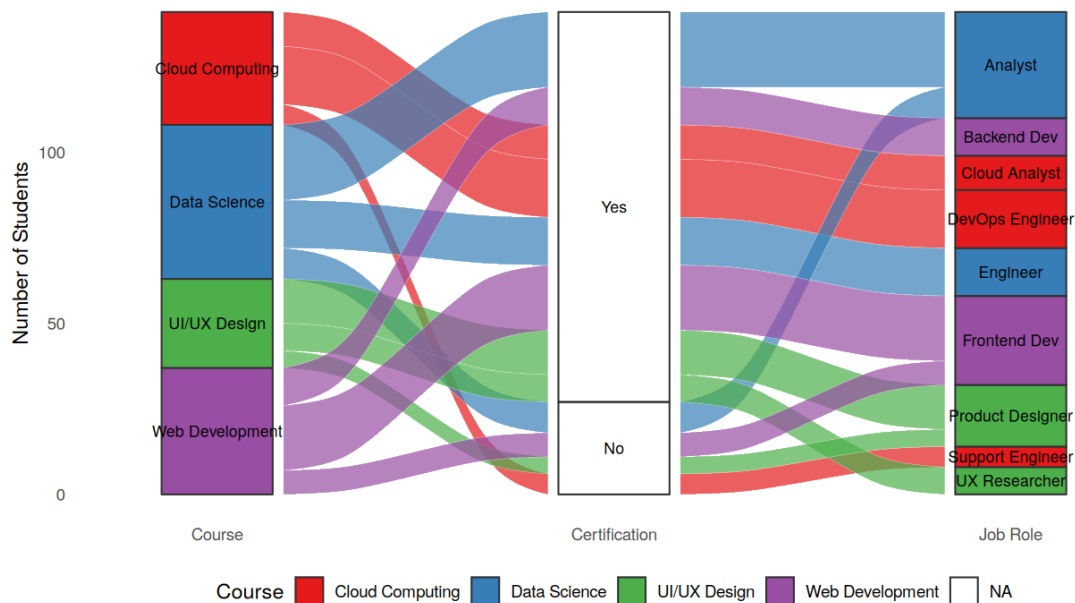


Figure 5.3 – Parallel sets alternative: categorical strata emphasised over flow curvature.

5. Comparing the Two Redesigns

The redesigned Sankey (Figure 5.2) uses smooth, curved alluvial ribbons that make it easy to visually trace how one course's cohort splits by certification status and lands in different job roles — the curvature itself communicates continuity of flow. The parallel sets alternative (Figure 5.3) uses wider, more rigid stratum blocks that emphasise direct category-to-category comparison at each stage, which can be useful for reading off group membership, but with this dataset it produces more crossing flow bands in the Certification stage and is slightly harder to trace end-to-end for a single course.

6. Which Visualization Better Communicates the Major Career Flows

The redesigned Sankey diagram communicates the major student career flows more effectively. Flow width directly encodes the number of students, the smooth ribbons make individual course-to-job-role journeys easy to trace, and distinct colors combined with labeled nodes remove the ambiguity present in the original chart. The parallel sets view remains a reasonable secondary/supporting diagram, but for a single clear narrative of 'which courses feed which job roles,' the Sankey layout is the stronger choice.